

Лекция 5

Тестовая документация

Тема большая и в ней мы рассмотрим как создавать чек-лист, тест-кейсы, отчеты о дефекте, а также поверхностно затронем тест-план, тестовую стратегию, отчеты по результатам тестирования.

На что стоит обратить внимание в первую очередь?

1. **Структура и обязательные атрибуты.** Большинство ошибок возникает по причине невнимательности, хотя каждый атрибут очень важен.
2. **Заголовки.** У каждого вида документации есть свои требования к заголовкам. Короткое название проверки из чек-листа не подходит для тест-кейса. Заголовок кейса не подходит для баг-репорта, так как несет совсем другую функцию.
3. **Особенности использования систем для хранения документации.** Вы должны научиться не только создавать документацию в специальных инструментах, но без проблем находить ее и понимать, на каких этапах SDLC она используется.

Шаблоны тест-кейсов - <https://victorz.ru/202001101079>

Чек-лист - как тестировать поиск? <https://habr.com/ru/post/577448/>

Чек-лист

Чек-лист (checklist) - набор идей: идей по тестированию, идей по разработке, идей по планированию и управлению — любых идей.

Самый простой пример чек-листа в отрыве от тестирования: список покупок, где покупка является проверкой, а чек-бокс - ее статус.

Обязательные части чек-листа

Шапка: содержит информацию о названии приложения, его версии, окружении, на котором проводится тестирование (версия ОС, браузера, эмулятора), ответственного за тестирование, дату тестирования

Тестируемые модули, submodule: например, регистрация, аутентификация авторизация

Список проверок: они должны отражать основную суть, без лишней детализации

Статус: информация о статусе прохождения проверки - пройдено/не пройдено (passed/failed)

Дополнительные части чек-листа

Ожидаемый результат: то, что мы ожидаем увидеть после запуска проверки согласно требованиям

Типы тестирования: к какому типу относится проверка?

Отчеты о дефекте: ссылки на отчеты о дефектах для прослеживаемости

Заметки: если нужно добавить комментарии

Также у проверок может быть **уникальный идентификатор** для прослеживаемости.

Фрагмент реального чек-листа

Project				Alexpress					
Date				06.08.2022					
Build				1.0					
Tester				Artsiom Rusau					
Environment				Windows 11 Version 22H2 Google Chrome103.0.5060.134	Safari	Firefox	Opera	Edge	
Module	Submodule	Element/ function	Summary	Status	Status	Status	Status	Status	
	Choose a location	Dropdown	All countries are loaded	passed					
			User can choose a country	failed					
			The country search is working						
			Cancel button						
			Input numbers > error message						
			Input special characters > error message						

Тест-кейс

Тестовый случай (test case) - набор предусловий, входных данных, действий (где применимо), ожидаемых результатов и постусловий, разработанных на основе тестовых условий.

В отличие от чек-листа, тест-кейс содержит в себе больше деталей: идентификатор, приоритет, заголовок, шаги и ожидаемые результаты.

Тест-кейсы объединяют в **тестовые наборы (test suites)** по одинаковым признакам: проверки для одного модуля, проверки для определенного типа тестирования, регрессионные наборы.

Атрибуты тест-кейса

Идентификатор (ID): уникальный номер, необходимый для прослеживаемости. В системах по управлению кейсами (TMS - test management system) проставляется автоматически

Приоритет (Priority): срочность и важность выполнения задачи

Требование (Requirement): ссылка на требование, для проверки которого служит кейс

Модуль (Module): название структурной части, в которой находится предмет тестирования

Заголовок (Title): Отражает суть проверки

Тестовые данные и предусловия (input data, test data, preconditions): информация о данных, которые необходимы для тестирования (данные для ввода, файлы с определенным расширением и размером и т.д.) + специальное состояние системы до начала тестирования (пользователь зарегистрирован, созданы объекты в базе данных и т.д.)

Шаги (Steps): последовательность действий для получения ожидаемого результата

Ожидаемые результаты (Expected results): по каждому шагу тест-кейса описывают реакцию приложения на действия, описанные в поле «шаги тест-кейса». Номер шага соответствует номеру результата.

Постусловия (Postconditions): возвращение системы в исходное состояние (удаление данных, пользователей, отключение виртуальной машины и т.д.)

Пример тест-кейса

ID	Summary	Pre-conditions	Test data	Steps	Expected results
1	The registration via email by non-existed user	1. The email was created 2. The user is not registered	Email: test@gmail.com	1. Go to https://best.aliexpress.com/ 2. Go to Account > Register button 3. Choose a location 4. Input email from test data 5. Input a password (8-20 symbols) 6. Click 'Create account' button 7. Check a mailbox 8. Copy a verification code and fill in a verification window 9. Click the 'Verify email' button 10. Log out 11. Try to log in with user credentials	1. The main page is opened 2. The registration window is opened 3. The dropdown with location is opened. User can choose a location 4. The email is displayed without error message 5. The password is displayed without error message. All validations are OK 6. The verification window is displayed. The 'Verify Email' button is disabled 7. The verification code was received 8. The 'Verify' button is active 9. The user was registered 10. The user is logged out 11. The user is logged in without error messages

Обязательными атрибутами для тест-кейса являются: идентификатор, приоритет, заголовок, шаги и ожидаемые результаты.

Все остальное используется в зависимости от конкретной задачи.

Отчет о дефекте

Дефект (defect, bug, problem, fault) — недостаток в компоненте или системе, способный привести к ситуации сбоя или отказа.

Отчёт о дефекте (defect report, bug report) — документ, описывающий и приоритизирующий обнаруженный дефект, а также содействующий его устранению.

Атрибуты отчета о дефекте

Идентификатор: уникальный номер отчета. Присваивается автоматически в багтрекингowej системе (BTS, bug tracking system)

Краткое описание (summary): отвечает на три вопроса: что, где, когда? Что произошло? Где это произошло? При каких условиях?

Подробное описание (description): информация о дефекте в развернутом виде + фактический и ожидаемый результат + ссылка на требование

Шаги по воспроизведению (steps to reproduce, STR)

Окружение (environment): информация о среде, на которой был обнаружен баг (версия ОС, браузера, мобильного устройства)

Важность (severity) - степень ущерба, который наносится проекту существованием дефекта. Серьезность для разрабатываемого ПО.

Критическая (critical) — существование дефекта приводит к масштабным последствиям катастрофического характера

Высокая (major) — существование дефекта приносит ощутимые неудобства многим пользователям в рамках их типичной деятельности

Средняя (medium) — существование дефекта слабо влияет на типичные сценарии работы пользователей, и/или существует обходной путь достижения цели

Низкая (minor) — существование дефекта редко обнаруживается незначительным процентом пользователей и (почти) не влияет на их работу

Срочность (priority) показывает, как быстро дефект должен быть устранён.

Наивысшая (ASAP, as soon as possible) срочность указывает на необходимость устранить дефект настолько быстро, насколько это возможно.

Высокая (high) срочность означает, что дефект следует исправить вне очереди, т.к. его существование или уже объективно мешает работе, или начнёт создавать такие помехи в самом ближайшем будущем.

Обычная (normal) срочность означает, что дефект следует исправить в порядке общей очерёдности. Такое значение срочности получает большинство дефектов.

Низкая (low) срочность означает, что в обозримом будущем исправление данного дефекта не окажет существенного влияния на повышение качества продукта.

Комментарий (comments, additional info) дополнительные данные о дефекте, возможность упомянуть исполнителя

Вложения (attachments) подтверждение дефекта: фото, видео, логи, тестовые данные, архивы и так далее

Также в дефектах обычно есть **следующие поля**:

1. Статус в жизненном цикле
2. Информация об исполнителе
3. Ссылки на связанные артефакты

Популярный вопрос про Priority и Severity

Один из популярных вопросов на интервью: привести примеры багов с высоким приоритетом и низкой серьёзностью и наоборот. Ознакомиться можно по следующей ссылке:

<https://testengineer.ru/sereznost-i-prioritet-bagov-v-chem-raznica/>

Высокий приоритет и низкая серьёзность

Такое сочетание бывает, когда баг на функционал влияет незначительно, но зато на пользовательский опыт влияет очень сильно. Также в эту категорию попадают баги, не влияющие на программу, но требующие исправления.

Вот пара примеров:

1. Кнопки перекрывают друг друга. Они кликабельны, но визуальное впечатление портится.
2. Логотип компании на главной странице содержит орфографическую ошибку. На функционал это вообще не влияет, но портит пользовательский опыт. Этот баг нужно исправить с высоким приоритетом, несмотря на то, что на продукт он влияет минимально.

Высокая серьезность и низкий приоритет

Такое сочетание бывает у багов, которые возникают в отдельных функциях программы. Эти баги не позволяют пользоваться системой, при этом обойти их невозможно. Но сами функции, содержащие эти дефекты, конечным потребителем используются редко.

Примеры:

1. Домашняя страница сайта ужасно выглядит в старых браузерах. Перекрывается текст, не загружается логотип. Это мешает пользоваться продуктом, поэтому серьезность бага высокая. Но так как очень мало пользователей открывают сайт при помощи устаревшего браузера, такой баг получает низкий приоритет.
2. Допустим, у нас есть приложение для банкинга. Оно правильно рассчитывает ежедневный, ежемесячный и ежеквартальный отчет, но при расчете годового возникают проблемы. Этот баг имеет высокую степень серьезности. Но если сейчас формирование годовой отчетности не актуально, такой дефект имеет низкий приоритет: его можно исправить в следующем релизе.

Серьезность дефекта (<u>Severity</u>)	Приоритет дефекта (<u>Priority</u>)
S1 Блокирующая (<u>Blocker</u>) Блокирующая ошибка, приводящая приложение в нерабочее состояние, в результате которого дальнейшая работа с тестируемой системой или ее ключевыми функциями становится невозможна. Решение проблемы необходимо для дальнейшего функционирования системы. S2 Критическая (<u>Critical</u>) Критическая ошибка, неправильно работающая ключевая бизнес логика, дыра в системе безопасности, проблема, приведшая к временному падению сервера или приводящая в нерабочее состояние некоторую часть системы, без возможности решения проблемы, используя другие входные точки. Решение проблемы необходимо для дальнейшей работы с ключевыми функциями тестируемой системой. S3 Значительная (<u>Major</u>) Значительная ошибка, часть основной бизнес логики работает некорректно. Ошибка не критична или есть возможность для работы с тестируемой функцией, используя другие входные точки. S4 Незначительная (<u>Minor</u>) Незначительная ошибка, не нарушающая бизнес логику тестируемой части приложения, очевидная проблема пользовательского интерфейса. S5 Тривиальная (<u>Trivial</u>) Тривиальная ошибка, не касающаяся бизнес логики приложения, плохо воспроизводимая проблема, малозаметная посредством пользовательского интерфейса, проблема сторонних библиотек или сервисов, проблема, не оказывающая никакого влияния на общее качество продукта.	P1 Высокий (<u>High</u>) Ошибка должна быть исправлена как можно быстрее, т.к. ее наличие является критической для проекта. P2 Средний (<u>Medium</u>) Ошибка должна быть исправлена, ее наличие не является критичной, но требует обязательного решения. P3 Низкий (<u>Low</u>) Ошибка должна быть исправлена, ее наличие не является критичной, и не требует срочного решения. Порядок исправления ошибок по их приоритетам: <u>High -> Medium -> Low</u>

- Наличие открытых дефектов P1, P2 и S1, S2, считается неприемлемым для проекта. Все подобные ситуации требуют срочного решения и идут под контроль к менеджерам проекта.
- Наличие строго ограниченного количества открытых ошибок P3 и S3, S4, S5 не является критичным для проекта и допускается в выдаваемом приложении. Количество же открытых ошибок зависит от размера проекта и установленных критериев качества.

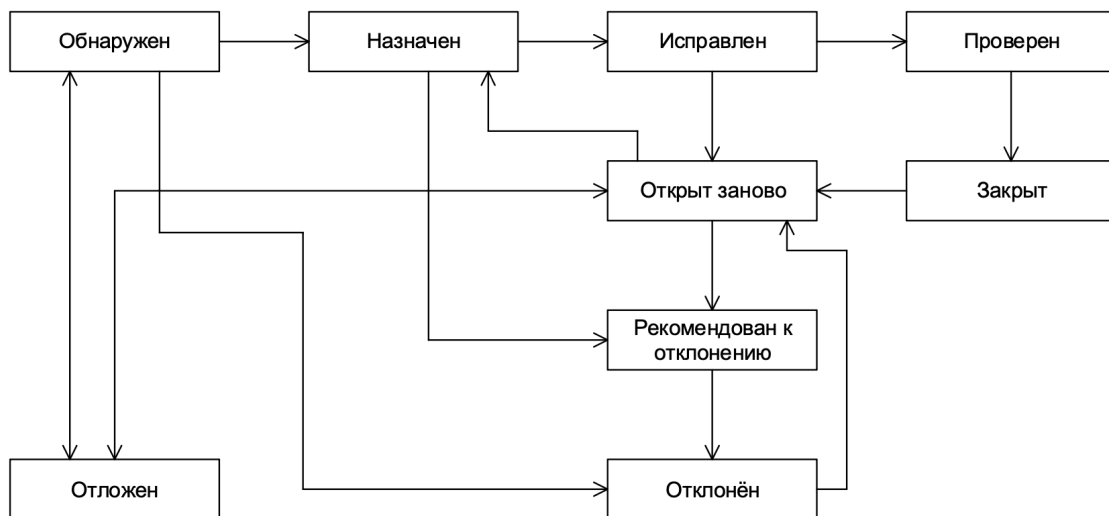


Рисунок 2.5.с — Жизненный цикл отчёта о дефекте с наиболее типичными переходами между состояниями

Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”



Также о стиле оформления отчета о дефекте можно узнать здесь <https://habr.com/ru/articles/358926/>

Как не стоит писать баг-репорты

The screenshot shows a bug report in the Yandex Tracker interface. The report title is "Инвентаризация. После нажатия на кнопку 'Начать' нельзя сканировать товары в текущую задачу". The status is "Открыт" (Open). The report includes a description of the issue, steps to reproduce, and the expected result. The interface also shows a sidebar with fields like "Тип" (Type), "Приоритет" (Priority), "Статус" (Status), "Дедлайн" (Deadline), "Автор" (Author), "Исполнитель" (Assignee), "Наблюдатели" (Observers), "Проект" (Project), and "Теги" (Tags).

Баг-репорты /

Инвентаризация. После нажатия на кнопку "Начать" нельзя сканировать товары в текущую задачу

↑ BUG-671597 **Открыт** Создано 10 часов Обновлено только что

Проверить Закрывать Действия

Предусловие: 1. Открыть приложение Инвентаризация

Шаги:

1. Создать задачу для сканирования товаров
2. Нажать на кнопку "Начать"
3. Нажать на сканере кнопку сканирования

Фактический результат: После нажатия на кнопку "Начать" нельзя сканировать товары в текущую задачу

Ожидаемый результат: После нажатия на кнопку "Начать" становится возможным сканировать товары в текущую задачу

Окружение: Apple iPod touch 32GB

Комментарии Файлы История

Сначала старые

Выбрать поля

Тип	Ошибка
Приоритет	Критичный
Статус	Открыт
Дедлайн	—
Автор	—
Исполнитель	Не назначен
Наблюдатели	Нет наблюдателей
Проект	—
Теги	—

Опытный тестировщик в ответ на такой документ скажет, что баг-репорт описан непонятно: что значит «нельзя сканировать»? **В результатах нужно описывать, что происходит, а не то, чего не происходит.** Не хватает информации о том, какие материалы нужно приложить к такому багу, скриншоты или скринкасты.

Баг-репорт от опытного джуна или ленивого мидла

Заголовок: [Инвентаризация] В начатой задаче при попытке сканирования товаров визуально ничего не происходит

Предусловия: Приложение Инвентаризация запущено и активно, открыта невыполненная задача для инвентаризации

Шаги:

1. Нажать кнопку «Начать».
2. Сканировать товар, присутствующий в задаче и еще не отсканированный.

Фактический результат: при попытке сканирования товара визуально ничего не происходит. В логах ошибка <Error...> (приложен лог с ошибкой).

Ожидаемый результат: сканирование проходит успешно, в логах нет ошибок, отсканированный товар записывается в открытую задачу согласно требованиям (ссылка на требования).

Окружение: Apple iPod touch 32 Gb.

Приоритет: критичный.

Баг-репорт от мидла или сеньора

Заголовок: [Инвентаризация] В начатой задаче сканирование товара не записывается в задачу, в логах ошибка <Error...>.

Предусловия: Приложение Инвентаризация запущено и активно, открыта невыполненная задача для инвентаризации.

Шаги:

1. Нажать кнопку «Начать».
2. Сканировать любой товар из задачи.

Фактический результат: при попытке сканирования товара сканер пищит (как и должен), но в задачу сканирование не записывается. В системных логах приложения ошибка <Error...> (приложен кусок лога с ошибкой). В серверных логах ошибка <Error...> (приложен кусок лога с ошибкой).

Ожидаемый результат: сканирование проходит успешно, в логах нет ошибок, отсканированный товар записывается в открытую задачу согласно требованиям (ссылка на требования).

Доп.информация: если отправить запрос о сканировании не с устройства, а через эмулятор, то ошибок не возникает, возвращается корректный ответ (приложены запрос и ответ и логи с эмулятора).

Окружение: Apple iPod touch 32 Gb, версия приложения 1.2.21.3, тестовый стенд INTG.

Приоритет: критичный.

источник примера:

<https://practicum.yandex.ru/blog/chto-takoe-bug-report-kak-ego-sostavit/>

Практическое задание

Перед вами тестовый [сайт-питомник](https://guru.qahacking.ru/index.php/magazin) с большим количеством багов.

<https://guru.qahacking.ru/index.php/magazin>

1. Составьте чек-лист или тест-кейс для проверки чего-либо на сайте.
2. Если ваш тестовая документация найдет баг необходимо задокументировать
3. Если ваша тестовая документация не находит баг выполнить 1 и 2 пункты заново.